

IMPACT: A Toolchain for Nonlinear Model Predictive Control Specification, Prototyping, and Deployment ^{*}

Alvaro Florez ^{*,1} Alejandro Astudillo ^{*,1} Wilm Decré ^{*}
Jan Swevers ^{*} Joris Gillis ^{*}

^{*} MECO Research Team, Dept. of Mechanical Engineering, KU Leuven.
Flanders Make@KU Leuven, 3001 Leuven, Belgium.
(e-mail: {alvaro.florez, alejandro.astudillovigoya, wilm.decre, jan.swevers, joris.gillis}@kuleuven.be).

Abstract: We present IMPACT, a flexible toolchain for nonlinear model predictive control (NMPC) specification with automatic code generation capabilities. The toolchain reduces the engineering complexity of NMPC implementations by providing the user with an easy-to-use application programming interface, and with the flexibility of using multiple state-of-the-art tools and numerical optimization solvers for rapid prototyping of NMPC solutions. IMPACT is written in Python, users can call it from Python and MATLAB, and the generated NMPC solvers can be directly executed from C, Python, MATLAB and Simulink. An application example is presented involving problem specification and deployment on embedded hardware using Simulink, showing the effectiveness and applicability of IMPACT for NMPC-based solutions.

Copyright © 2023 The Authors. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Keywords: Model Predictive Control, Software Toolchain, Prototyping, Deployment

1. INTRODUCTION

The burgeoning computational power of off-the-shelf processing units has enabled the use of model predictive control (MPC) for applications that involve increasingly complex systems, need to comply with constraints and need to account for several, sometimes conflicting, performance objectives. However, development and implementation of nonlinear MPC (NMPC) comes with a high engineering cost, and the availability of flexible, easy-to-use tools for rapid prototyping and deployment of NMPC solvers is insufficient. This has limited the wide adoption of NMPC, e.g., for complex and fast mechatronic applications.

Central to an MPC control scheme is the repeated online numerical solution of a parametric optimal control problem (OCP). Important exceptions such as GRAMPC (Englert et al., 2019) notwithstanding, most efficient solvers rely on a direct transcription method to transform the OCP into a nonlinear program (NLP). In recent years, a rich landscape of efficient quadratic programming (QP) solver implementations has emerged. They are valuable as a subroutine in an important class of NLP solvers, as well as directly in linear MPC, and real-time iteration schemes for NMPC.

No combination of OCP method, NLP solver and QP solver is applicable, let alone optimal, across a wide range of control applications. This situation has given rise to

a rich solver landscape². Some of these solvers are akin to a framework: offering some degree of vertical integration, i.e., going all the way from high-level easy-to-use OCP modeling down to low-level efficient deployable code. Examples of efficient frameworks are: AutoGenU (Katayama and Ohtsuka, 2020), providing an environment to formulate, solve, simulate and code generate MPC within Jupyter notebooks, featuring a generalized minimal residual method implemented in C++ — OpEn (Sopasakis et al., 2020), focusing on embedded nonconvex optimization using the proximal averaged Newton-type method for optimal control (PANOC) and the augmented Lagrangian method, implemented in Rust and providing interfaces to Python and MATLAB — Acados (Verschuere et al., 2021) and MATMPC (Chen et al., 2019), offering SQP-type methods with Simulink integration — ParNMPC (Deng and Ohtsuka, 2018), offering an interior point solver with parallelization capabilities in MATLAB — MATLAB MPC toolbox (Mathworks, 2019), offering a graphical interface and tight integration with Simulink.

The main drawback of existing frameworks is that they are monolithic rather than modular: *they restrict the user to a single or limited set of numerical algorithms*, conflicting with the need of practitioners to prototype, e.g. figuring out the best solver and settings combination for a specific OCP. Furthermore, each provides a *specific way to model OCPs and to communicate with the generated solution*, which makes switching between frameworks, for testing and prototyping, a cumbersome process. Lastly, a *lack of*

^{*} This work was supported by the project Flanders Make SBO DIRAC: “Deterministic and Inexpensive Realizations of Advanced Control”. Flanders Make is the Flemish strategic research centre for the manufacturing industry.

¹ A. Florez and A. Astudillo contributed equally to this work.

² See https://github.com/meco-group/dynamic_optimization_inventory

high-level modeling means that the user is burdened with writing in canonical forms and supplying derivatives.

To address the aforementioned issues, we present IMPACT, an open-source toolchain that aims to facilitate the workflow of specification, prototyping and deployment of NMPC. It offers *vertical integration, while providing modularity, with minimal runtime overhead*. The main features of IMPACT are described as follows:

- it provides a unified high-level application programming interface (API) to model OCPs;
- it is cross-lingual (written in Python and provides auto-generated³ MATLAB bindings) and cross-platform;
- it offers a range of direct transcriptions methods that are paired to state-of-the-art QP/NLP solvers via CasADi plugins such as HPIPM (Frison and Diehl, 2020), PROX-QP (Bambade et al., 2022), IPOPT (Wächter and Biegler, 2006), FSLP (Kießling et al., 2022);
- it offers alternatives to built-in direct transcription by interfacing OCP solvers/frameworks via Rockit plugins – at the moment Acados, GRAMPC and FATROP (Vanroye et al., 2022);
- it exports problem-specific MPC libraries with a unified C API, useful for rapid prototyping and deployment on simulated and real environments;
- it exports a problem-specific Simulink block making use of the exported MPC library.

IMPACT is released under the LGPLv3 license and its source code is available at <https://gitlab.kuleuven.be/meco-software/impact>. It is built on top of the powerful tools for optimization and optimal control CasADi (Andersson et al., 2019) and Rockit (Gillis et al., 2020).

The remainder of this paper is organized as follows. Section 2 presents preliminary concepts on the formulation and transcription of the OCP underpinning MPC. In Section 3, details on the overall structure and workflow of IMPACT are given. An application example using IMPACT is described in Section 4. Finally, we close the paper with conclusions and future work remarks.

2. OPTIMAL CONTROL PROBLEM

For a time horizon $t \in [t_0, t_f]$, system state vector $\mathbf{x}(t) \in \mathbb{R}^{n_x}$, input vector $\mathbf{u}(t) \in \mathbb{R}^{n_u}$, algebraic state vector $\mathbf{z}(t) \in \mathbb{R}^{n_z}$, and parameter vector $\mathbf{p} \in \mathbb{R}^{n_p}$, in IMPACT we consider general OCP formulations of the canonical form

$$\underset{\mathbf{x}, \mathbf{u}, \mathbf{z}}{\text{minimize}} \quad \int_{t_0}^{t_f} V(\mathbf{x}(t), \mathbf{u}(t), \mathbf{p}, t) dt + V_f(\mathbf{x}(t_f), \mathbf{p}) \quad (1a)$$

$$\text{subject to} \quad B(\mathbf{x}(t_0), \mathbf{x}(t_f), \mathbf{p}) \leq 0, \quad (1b)$$

$$\dot{\mathbf{x}}(t) = \xi(\mathbf{x}(t), \mathbf{u}(t), \mathbf{z}(t), \mathbf{p}), \quad t \in [t_0, t_f], \quad (1c)$$

$$\Gamma(\mathbf{x}(t), \mathbf{u}(t), \mathbf{z}(t), \mathbf{p}) = 0, \quad t \in [t_0, t_f], \quad (1d)$$

$$h(\mathbf{x}(t), \mathbf{u}(t), \mathbf{p}) \leq 0, \quad t \in [t_0, t_f], \quad (1e)$$

where $V(\cdot)$ and $V_f(\cdot)$ in (1a) are smooth nonlinear functionals that define Lagrange and Mayer terms, respectively, $\mathbf{p}$ is a parameter vector that typically contains a vector of state measurements or estimations $\mathbf{x}_{\text{meas}} \in \mathbb{R}^{n_x}$, (1b) defines a boundary constraint that typically takes the form of

$\mathbf{x}(t_0) = \mathbf{x}_{\text{meas}}$, (1c) and (1d) define a system of differential-algebraic equations (DAE) representing the system model, while (1e) represent general path constraints. Note that the inclusion of the algebraic state vector $\mathbf{z}$ and the algebraic equation (1d) is not required and can be omitted for systems represented solely by ordinary differential equations (ODE). The OCP specified by the user in IMPACT (see Section 3.3) is automatically transformed into the canonical form (1).

Direct transcription of (1) leads to a finite-dimensional optimization problem of the form

$$\min_{\mathbf{w}} f(\mathbf{w}) \quad \text{s.t.} \quad g(\mathbf{w}) = 0, \quad h(\mathbf{w}) \leq 0, \quad (2)$$

where $\mathbf{w} \in \mathbb{R}^{n_w}$ is the vector of decision variables, defined while considering $N := T/\delta_t \in \mathbb{N}$ discretization points within the prediction horizon $T := (t_f - t_0) \in \mathbb{R}_{>0}$ and a sampling time $\delta_t \in \mathbb{R}_{>0}$. The first- and second-order derivatives arising in the optimality conditions of (2) are typically large and sparse.

3. TOOLCHAIN DESCRIPTION

This section presents an overview of the structure of the IMPACT toolchain. It first gives a description of the fundamental dependencies of the toolchain, and then presents the elements of the toolchain and a workflow from MPC specification to deployment. Finally, it details the use of the unified API to communicate with the MPC solvers.

3.1 Fundamental Dependencies

The fundamental software dependencies required for installing and using IMPACT are: CasADi, Rockit, and the Python modules pyyaml and lxml.

IMPACT relies on CasADi to handle expressions that model the objective and constraints as expression graphs, and to allow the saving/loading of functions and algorithms and their transfer across Python and MATLAB.

If instructed by the user, IMPACT relies on Rockit to perform a chosen OCP transcription method. At the moment, multiple-shooting, single-shooting, direct collocation, and a B-spline method are supported. Derivatives of the resulting NLP objective and constraints are obtained through CasADi's sparsity-exploiting algorithmic differentiation (AD), optionally made more efficient by C-code generation.

Rockit further provides interfaces to third-party OCP frameworks such as GRAMPC, Acados and FATROP. Initially meant for offline usage, these interfaces are being refactored to use the CasADi codegen C API, such that they can be embedded in CasADi expressions and are compatible with C-code generation. Lastly, Rockit relies on CasADi for importing functional mock-up units (FMU) for input/output representations of systems.

The libraries pyyaml and lxml are used within IMPACT to handle YAML and XML files required in the process of loading system models and generating Simulink blocks when exporting the solution, respectively.

3.2 Structure of the IMPACT toolchain

A graphical illustration of the overall structure of the IMPACT toolchain is shown in Fig. 1. CasADi and Rockit

³ See <https://gitlab.kuleuven.be/meco-software/python-matlab>

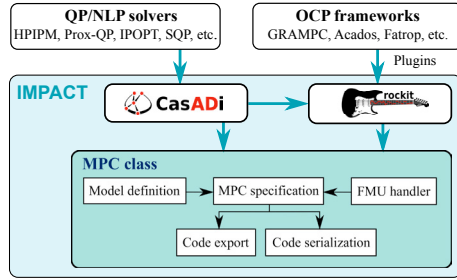


Fig. 1. Overview architecture of the IMPACT toolchain showing the interaction between modules.

play a vital role within IMPACT by providing essential capabilities to the toolchain regarding, e.g., expression handling, and interfacing third-party solvers. In fact, the dependency of IMPACT on CasADi and Rockit, and the inherent modularity that such dependencies represent, allow the direct and transparent availability of new solvers in IMPACT whenever an interface to such solvers is added to either CasADi or Rockit.

Apart from CasADi and Rockit, IMPACT is composed by the MPC class, which is instantiated by the user to use all the functionalities of IMPACT. This class is divided into five sub-modules, which are described as follows.

1) Model definition: IMPACT provides the user with two ways of defining (nonlinear) system models to be used within the OCP definition: (i) manually defining state and control variables in addition to differential and algebraic equations, as done in Rockit or (ii) using YAML files – i.e., which are human-readable files for data serialization – by using the `add_model()` method of the MPC class. By using YAML files, we aim to standardize the definition of models based on ODEs or DAEs for their use in any software tool that is compatible with YAML. This model representation allows the use of both inline definition of equations and external serialized CasADi functions, i.e., `*.casadi` files, as shown respectively in the two YAML snippets of a pendulum system that follow.

```

equations:
  inline:
    ode:
      phi: dphi
      dphi: L*cos(phi)*sin(phi) ...
  differential_states:
    - name: phi
    - name: dphi
  controls:
    - name: F
  constants:
    inline:
      L: 2
equations:
  external:
    type: casadi_serialized
    file_name: ode.casadi
  differential_states:
    - name: phi
    - name: dphi
  controls:
    - name: F

```

2) FMU handler: This module allows the user to load FMUs – an industry standard to define containers that represent black-box (input/output) models – as CasADi functions to be used within the OCP definition. By calling the `add_simulink_fmu()` method of the MPC class, the FMU is wrapped into a C++ file, which is then compiled and loaded back as a CasADi external function. When the FMU provides derivatives (forward derivatives per FMI standard 2.0), these can be used. Otherwise, CasADi uses finite differences. In a further stage, FMU handling should be absorbed into the YAML file approach.

3) MPC specification: This module inherits the functionality of the OCP class of Rockit to enable the specification of the OCPs underpinning MPC. With the OCP

class, Rockit allows the definition of (multi-stage) OCPs by (i) managing symbolic variables for system states $\mathbf{x}$, system inputs $\mathbf{u}$, and algebraic states $\mathbf{z}$, (ii) associating dynamics to $\mathbf{x}$ and algebraic equations to $\mathbf{z}$ as in (1c) and (1d), (iii) composing the objective function (1a) by adding functions evaluated at different instants within the OCP horizon, and (iv) setting path or boundary constraints to expressions or variables as in (1b) and (1e). When choosing a third-party OCP plugin, restrictions on the problem specification may apply. The MPC specification is also used for result post-processing capabilities which allow the user, for instance, to retrieve the values of specific variables or expressions in the OCP solution, or to interpolate the solution on a refined grid of integration points.

4) Code serialization: This module allows the serialization of the expression graph created by CasADi to define the MPC solver. This allows to save an MPC solver instantiation (including the functions that define the OCP) into a file that can be then loaded from Python, MATLAB or C++. This module relies on the `save()` and `load()` methods of the MPC class. In a similar fashion, instances of the IMPACT Python module can also be saved and loaded.

5) Code export: By calling the `export()` method of the MPC class, the user generates multiple artifacts that allow the prototyping and deployment of a solver within an MPC implementation. The main artifacts depicted in Figure 2 are (i) an IMPACT library callable through the IMPACT C API, (ii) a numerical backend library containing a concrete low-level problem formulation coupled to a solver that is either included as fully self-contained C code, linked in with a dependency on the C++ CasADi runtime library and chosen solver plugin such as IPOPT, or linked in as third-party dependencies such as OSQP or GRAMPC, and (iii) an MPC-ready IMPACT Simulink block.

The IMPACT library is accompanied by Python bindings. More details can be found in Section 3.4.

Self-contained C code is dependency-free and requires only a C compiler to be deployed in any compatible target. It requires every element of the MPC solver instantiation to be compatible with code-generation – e.g., if the user selects IPOPT, which cannot be code-generated, no self-contained C code of the MPC solver can be generated.

The IMPACT Simulink block is based on a code-generated custom C S-function. IMPACT generates a `.slx` container based on the C API and linked to the generated C files, and can be directly used within a Simulink model. Simulink has become a de-facto tool for simulation and prototyping of control systems. This artifact allows IMPACT users to easily prototype their designed MPC with no extra steps. Moreover, if the linked C file is self-contained, the generated Simulink block would be compatible with the Simulink coder. This way, the MPC-ready IMPACT Simulink block could be code-generated as part of a larger Simulink model and deployed into real-time targets without additional engineering effort. Compatibility with Simulink coder is also expected when third-party dependencies are open-source and C based, but this workflow has not yet been validated.

The three artifacts include CasADi functions for the MPC solver (with parametric and hot-start inputs), forward

simulation of the system dynamics, DAEs, performance objective, among others. In addition, they output statistics of the solution such as solution time, status, optimality of the solution and number of iterations. These provide the user with a great deal of flexibility to implement and prototype the MPC solution within a simulated or real control loop. Note that once the artifacts are generated, the only values that can be modified within the problem are the parameters $\mathbf{p}$.

3.3 Toolchain Workflow

This section has given details on the architecture of IMPACT and its dependencies. It is now necessary to explain the workflow proposed within IMPACT for the user to specify an MPC, prototype its solution, and deploy it. A general overview of the workflow is presented in Fig. 2.

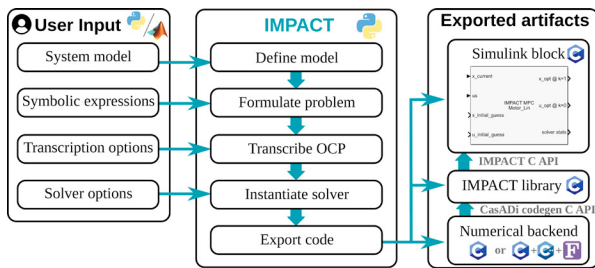


Fig. 2. Overview of the workflow of the IMPACT toolchain.

The workflow in the IMPACT toolbox is described as follows by means of a simple example involving the control of a pendulum. A description of each step is provided followed by the code snippet associated with it.

First of all, the user must import the IMPACT module and instantiate an object of the MPC class. The argument $T=2.0$ sets the prediction horizon T to a fixed value. However, T could be considered as a decision variable within a free time problem by passing $T=\text{FreeTime}(2.0)$ as argument, where 2.0 is an initial guess for this variable.

```

from impact import *
mpc = MPC(T=2.0)

```

Afterwards, the system model should be declared. As mentioned in Section 3.2, this can be done by manually writing the ODE or DAE – i.e., using instances of states and inputs within the MPC class – or, as shown in the code snippet below, by loading a YAML file – as in the example of a pendulum system presented in Section 3.2.

```
pendulum = mpc.add_model('pendulum', 'pendulum.yaml')
```

Once the model has been defined, the user can instantiate the elements of the vector of parameters $\mathbf{p}$, such as the current state ($\mathbf{x}_0$), a desired final state ($\mathbf{x}_f$), and the weights ($\mathbf{Wt}$) to be used in the objective.

```

x_0 = mpc.parameter('x_0', pendulum.nx)
x_f = mpc.parameter('x_f', pendulum.nx)
Wt = mpc.parameter('Wt', 2)

```

In a high-level way, the objective and constraints are then defined by using the methods `mpc.add_objective()` and `mpc.subject_to()` directly using state and control names. Note that, by using `mpc.at_t0()` or `mpc.at_tf()` the user can specify boundary constraints.

```

# Objective
mpc.add_objective(mpc.integral(Wt[0]*pendulum.F**2 + Wt[1]*pendulum.dphi**2))
# Boundary constraints
mpc.subject_to(mpc.at_t0(pendulum.x) == x_0)

```

```

mpc.subject_to(mpc.at_tf(pendulum.x) == x_f)
# Path constraints
mpc.subject_to(-2 <= (pendulum.F <= 2))

```

The nonlinear optimization solver to be used is defined by means of the `mpc.solver()` method, for which the user provides several options depending on the selected solver. In this case, the SQP method of CasADi has been selected.

```
mpc.solver('sqpmethod', options={...})
```

The transcription method is defined either by using a direct method provided by Rockit or an external method provided by a plugin to an optimization tool such as GRAMPC. Here, N corresponds to the number of discretization points N , while `intg='rk'` sets a 4th-order Runge Kutta integrator to discretize the system dynamics.

```

# Direct method (MultipleShooting, SingleShooting, etc.), option 1
method = MultipleShooting(N=40, intg='rk')
# External method ('grampc', 'fatrop', etc.), option 2
method = external_method('grampc', N=40, grampc_options=...)
# Actually set the selected method within the MPC environment
mpc.method(method)

```

The user can now set the values of the parameters $\mathbf{p}$ and execute the solver directly in Python.

```

mpc.set_value(x_0, [0.5, 0, 0, 0]) # Set parameters
mpc.set_value(x_f, [0, 0, 0, 0])
mpc.set_value(Wt, [1, 1])
solution = mpc.solve() # Solve the OCP

```

Finally, the user can generate the three artifacts explained in Section 3.2 by executing the following code.

```
mpc.export("pendulum")
```

For rapid prototyping of the exported code the user can: (i) run the generated `hello_world` examples in C or Python which use the C API, or (ii) use the generated Simulink block from the generated `.slx` file in a Simulink model. Since we are using the SQP method of CasADi, which is compatible with code-generation, in this example we could deploy the code-generated code or the Simulink block in a real-time target architecture.

3.4 IMPACT C API

The IMPACT C API defines a unified API to use the MPC solver exported by IMPACT. It represents the core artifact of the code export functionality and is a layer of abstraction that makes the implementation agnostic to the content of the generated MPC solver, e.g., self-contained C code with C solver, C++ code that still depends on CasADi and/or third-party solvers. It provides rich functions to set the inputs of the MPC solver (parameters) and get the values of variables from the optimized solution. For the sake of completeness, we present below a minimum example of the use of the IMPACT C API for the pendulum example presented in Section 3.3.

```

impact_struct* m = impact_initialize();
impact_set(m, "x_0", IMPACT_ALL, 0, x_meas, IMPACT_FULL); // Set a parameter
impact_solve(m); // Solve a single OCP
// Get optimal input for first time instance
impact_get(m, "u_opt", IMPACT_ALL, 0, U, IMPACT_FULL);

```

Here, `impact_initialize()` instantiates the exported IMPACT library and returns a pointer m to it. By using the `impact_set()` method, the parameter $\mathbf{x}_0$ is set with a user-defined value $\mathbf{x}_{\text{meas}}$. The third argument `IMPACT_ALL` tells IMPACT that all the elements of the specified parameter are set. The user can use

"pendulum.phi" instead of `IMPACT_ALL` to set the parameter corresponding to the angle `phi`, for instance. The fourth argument 0 defines the time instance within the horizon for which the parameter is set. Another option is `IMPACT_EVERYWHERE`, which sets the parameter for the whole horizon. The sixth argument combines flags representing repetition (`IMPACT_FULL`, `IMPACT_HREP`) and data ordering (`IMPACT_ROW_MINOR`, `IMPACT_COLUMN_MAJOR`). The `impact_solve()` function executes the MPC solver. Then, an element of the solution – i.e., the optimal control input $\mathbf{u}_{\text{opt}}$ – is retrieved and assigned to a variable $\mathbf{U}$ by using `impact_get()`, with the arguments mirroring `impact_set()`. This example shows the solution of one OCP. To implement (N)MPC, the `impact_set()`, `impact_solve()`, and `impact_get()` methods are executed in a loop. More methods are defined in the API, e.g., to retrieve statistics and perform debugging.

4. APPLICATION EXAMPLE USING IMPACT

This section demonstrates the use and deployment of MPC using `IMPACT`. We control the point-to-point motion (angular position) of a DC motor using a Speedgoat SN7233 real-time target machine. The control scheme estimates a constant disturbance that represents unmodeled dynamics – e.g., input disturbances and static friction –, counteracts its effect, and achieves offset-free positioning.

4.1 System Description

The hardware has two components: the DC motor and the real-time target machine that executes the MPC algorithm and interfaces with the motor driver and a rotary encoder.

The shaft of the DC motor is connected to a load and the rotary encoder, which provides a measurement $\mathbf{y}$ of the angular position of the rotor $\theta \in \mathbb{R}$. The motor has a driver that receives an analog voltage $v_{\text{motor}} \in [-10, 10]$ from the digital-to-analog converter (DAC) of the Speedgoat. For a state vector $\mathbf{x} := [\theta, \dot{\theta}]^T \in \mathbb{R}^2$ and a control input $\mathbf{u} := v_{\text{motor}}$, a second order linear model for the motor

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad (3)$$

has been experimentally identified using the `LCToolbox` (Verbandt et al., 2018). This model contains one integrator and one real pole which models the linear motor friction, but does not consider the static friction component.

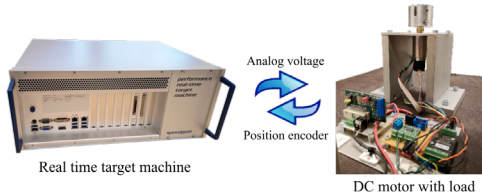


Fig. 3. Overview of the system.

The Speedgoat SN7233 is a real-time target machine, programmed via Simulink, that allows the deployment of controllers on, e.g., multi-core CPUs, FPGAs and PLCs, and facilitates rapid controller prototyping and hardware-in-the-loop simulations through a smooth integration with Simulink Real-Time. The target runs the MPC algorithm – based on an `IMPACT` Simulink block –, applies an analog voltage v_{motor} to the motor, and reads the digital signals from the rotary encoder, as depicted in Fig. 3.

4.2 Problem Definition

To achieve an accurate positioning of the rotor, i.e., counteracting the effect of the model-plant mismatch and a constant input disturbance $\mathbf{d} \in \mathbb{R}$, we define an offset-free MPC (Rawlings et al., 2017, Section 1.5). In addition to the solution of OCP (1), this methodology involves (i) augmenting the system dynamics within the estimation algorithm with the dynamics of $\mathbf{d}$, and (ii) solving an optimization problem – the steady-state problem or target selector – that sets references for $\{\mathbf{x}, \mathbf{u}\}$ in the OCP (1) based on the estimated states and disturbance, to compensate the disturbance effects while tracking a reference. The offset-free MPC structure is synthesized in Fig. 4.

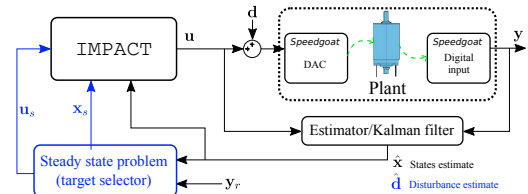


Fig. 4. Diagram of an offset-free MPC implementation, with differences with respect to traditional MPC in blue, and physical connections to the motor in green.

Before presenting details on the definition of OCP (1) for this application, let us describe the estimator and steady-state problem. The estimator – i.e., a Kalman filter – outputs an estimate $\hat{\mathbf{x}}_k$ of an augmented state vector $\tilde{\mathbf{x}}_k := [\mathbf{x}_k^T \mathbf{d}_k^T]^T$ whose dynamics are based on a discretized representation of (3) and the assumption of a constant disturbance. The steady-state problem corresponds to an optimization problem that considers the reference $\mathbf{y}_r$ and the estimated disturbance $\hat{\mathbf{d}}$ to define set-points $\{\mathbf{x}_s, \mathbf{u}_s\}$ for $\{\mathbf{x}, \mathbf{u}\}$ in OCP (1). For this example, the solution of such problem can be found analytically as the solution of a system of linear equations. The reader is referred to Rawlings et al. (2017) for more details on this formulation.

Instead of minimizing the output error $\mathbf{y} - \mathbf{y}_r$, OCP (1) aims to minimize the deviation between the set-points $\{\mathbf{x}_s, \mathbf{u}_s\}$ and the system variables $\{\mathbf{x}, \mathbf{u}\}$. Therefore, the objective (1a) is defined by $V(\cdot) := \|\mathbf{x}(t) - \mathbf{x}_s\|_Q^2 + \|\mathbf{u}(t) - \mathbf{u}_s\|_R^2$ and $V_f(\cdot) := \|\mathbf{x}(t_f) - \mathbf{x}_s\|_{Q_f}^2$, where $Q, R, Q_f \geq 0$ are weight matrices. In the boundary constraint (1b), $\mathbf{x}_{\text{meas}} = \hat{\mathbf{x}}$, i.e., excluding the estimated disturbance. The dynamics (1c) are described by the ODE (3). Since the system does not feature algebraic equations, (1d) is omitted. Boundary constraints (1e) are given by the lower and upper bounds on v_{motor} , such that $-10 \leq \mathbf{u}(t) \leq 10 \forall t \in [t_0, t_f]$.

4.3 Implementation Details

To implement the `IMPACT` Simulink block that solves OCP (1) we follow the workflow outlined in Section 3.3. We use $N = 50$, $\delta_t = 3.33$ [ms] and $T = 166.66$ [ms]. Model (3) is defined in a YAML file and loaded to the MPC object. Five parameters are defined, namely, $\mathbf{x}_{\text{meas}}$, $\mathbf{x}_s$, $\mathbf{u}_s$, Q , R and Q_f . The objective and constraints of OCP (1) are defined as in Section 4.2. We use the active set-based QP solver `QRQP` (Andersson and Rawlings, 2018) in `CasADi` and multiple shooting as transcription method

with a 4th-order Runge-Kutta integrator for system discretization. The MPC solver is exported to generate the Simulink block. This block is loaded into a Simulink model where the estimator and the steady-state problem are implemented, and where the Speedgoat input/output blocks are loaded. The IMPACT block is connected to the other blocks as in Fig. 4 to close the loop. The reference is $\mathbf{y}_r(t) = -3$ [rad], $\forall t \in [1, 3]$, and zero otherwise. While $\mathbf{d} = 0.5$ [V] $\forall t \geq 8$.

4.4 Results

Once the Simulink model has been set, we call the real time execution operation of Simulink Real-Time, which code-generates it by using the Simulink coder and then executes it in the Speedgoat. The results of the offset-free MPC implementation are compared against a traditional MPC implementation, i.e., without disturbance estimation and execution of the steady-state problem. Fig. 5 shows the evolution of the angular position θ and the voltage input $\mathbf{u}$ with both implementations. Here, the offset-free MPC can follow the reference with zero offset, while the traditional MPC deviates from the reference before (due to the effects of model-plant mismatch) and after the application of the disturbance at $t = 8$.

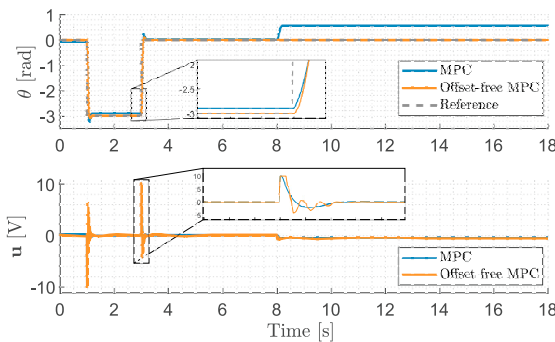


Fig. 5. Evolution of the angular position θ and the voltage input $\mathbf{u}$ during the application execution.

IMPACT eased MPC specification, testing and deployment, allowing to quickly prototype the problem and test several solvers by using the generated artifacts. In addition, the IMPACT Simulink block could be easily integrated within larger models for simulation or deployment on real hardware. Although this example features a linear system, IMPACT allows the definition of systems represented by both linear or nonlinear equations. Other tested applications with IMPACT include the deployment of NMPC in a Speedgoat target and a Beckhoff TwinCAT Embedded PC for a point-to-point motion application with a parallel SCARA robot. In the future, IMPACT will be tested in a DSpace Controller board, and extended to be used in FPGAs and GPUs.

5. CONCLUSIONS

In this paper we presented IMPACT, a toolchain for nonlinear model predictive control that reduces the engineering complexity of the different steps from problem specification to MPC solution deployment. The different modules that compose the toolchain, their dependencies and their usage workflow were described. In the wide space of academic software, IMPACT stands out by offering a unified

cross-lingual high-level way to model OCPs, and a unified C API to communicate with exported MPC controllers. Future directions include the implementation of a method to allow the C API to be exported as an FMU. We will also validate the toolchain in more complex scenarios involving complex robotic systems and self-driving cars.

REFERENCES

- Andersson, J.A., Gillis, J., Horn, G., Rawlings, J.B., and Diehl, M. (2019). Casadi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1), 1–36.
- Andersson, J.A. and Rawlings, J.B. (2018). Sensitivity analysis for nonlinear programming in CasADi. *IFAC-PapersOnLine*, 51(20), 331–336.
- Bambade, A., El-Kazdadi, S., Taylor, A., and Carpentier, J. (2022). PROX-QP: Yet another Quadratic Programming Solver for Robotics and beyond. In *RSS 2022 - Robotics: Science and Systems*. New York, United States.
- Chen, Y., Bruschetta, M., Picotti, E., and Beghi, A. (2019). MATMPC - a MATLAB based toolbox for real-time nonlinear model predictive control. In *2019 18th European Control Conference (ECC)*. IEEE.
- Deng, H. and Ohtsuka, T. (2018). A parallel code generation toolkit for nonlinear model predictive control. In *Proceedings of the 57th IEEE Conf. on Decision and Control*, 4920–4926. Miami, USA.
- Englert, T., Völz, A., Mesmer, F., Rhein, S., and Graichen, K. (2019). A software framework for embedded nonlinear model predictive control using a gradient-based augmented lagrangian approach (GRAMPC). *Optimization and Engineering*, 20(3), 769–809.
- Frison, G. and Diehl, M. (2020). Hpipm: a high-performance quadratic programming framework for model predictive control. doi:10.48550/arxiv.2003.02547.
- Gillis, J., Vandewal, B., Pipeleers, G., and Swevers, J. (2020). Effortless modeling of optimal control problems with rokit. In *39th Benelux Meeting on Systems and Control, Date: 2020/03/10-2020/03/12, Location: Elspeet, The Netherlands*.
- Katayama, S. and Ohtsuka, T. (2020). Automatic code generation tool for nonlinear model predictive control with jupyter. *IFAC-PapersOnLine*, 53(2), 7033–7040.
- Kiessling, D., Zanelli, A., Nurkanović, A., Gillis, J., Diehl, M., Zeilinger, M., Pipeleers, G., and Swevers, J. (2022). A feasible sequential linear programming algorithm with application to time-optimal path planning problems. In *Proceedings of 61st IEEE Conference on Decision and Control*. Cancun, Mexico.
- Mathworks (2019). Model Predictive Control Toolbox. URL <https://cutt.ly/mpc-matlab>.
- Rawlings, J., Mayne, D., and Diehl, M. (2017). *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing.
- Sopasakis, P., Fresk, E., and Patrinos, P. (2020). OpEn: Code generation for embedded nonconvex optimization. *IFAC-PapersOnLine*, 53(2), 6548–6554.
- Vanroye, L., Vochten, M., De Schutter, J., and Decré, W. (2022). Fast optimization-based trajectory generation for online applications. In *Workshop on Intelligent Control Methods and Machine Learning Algorithms for Human-Robot Interaction and Assistive Robotics, ICRA 2022*. Philadelphia, USA.
- Verbandt, M., Jacobs, L., Turk, D., Singh, T., Swevers, J., and Pipeleers, G. (2018). Linear control toolbox - supporting b-splines in lpv control. *Mechatronics (Oxford)*, 52, 78–89.
- Verschueren, R., Frison, G., Kouzoupis, D., Frey, J., van Duijkeren, N., Zanelli, A., Novoselnik, B., Albin, T., Quirynen, R., and Diehl, M. (2021). acados—a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 14(1), 147–183.
- Wächter, A. and Biegler, L.T. (2006). On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1), 25–57.